Report on Homework 3

——Deep Learning

刘滨瑞

2021012579　未央-水木 12

2024 年 1 月 10 日

1 Transformer

运行环境为：Ubuntu 22.04.3 LTS，Python 3.9.18，PyTorch 2.1.2。

1.1 Task1: Standard Transformer Model

参考原论文和课件中给出的相关公式，我们补全了标准的 Transformer 模型，源代码参见./Transformer/layers/attention.py 中的 MultiHeadAttention 类和 multi_head_attention 函数。[1]

训练过程中，模型在验证集、测试集和训练集上的 loss 变化曲线如图 1 所示。可以看出，模型收敛较快，6 个 epochs 时模型在验证集上的 loss 已不再继续降低。

如图 2 所示，我们展示了部分的预测结果，发现模型进行回归预测的效果并不是特别理想。推测可能的原因有两点。一是数据量较少，特征与预测值间的相关性不高，因此回归预测任务的难度较大；二是在我们使用的 Transformer 模型中，层数、注意力头数，隐藏层参数量都很少，模型的拟

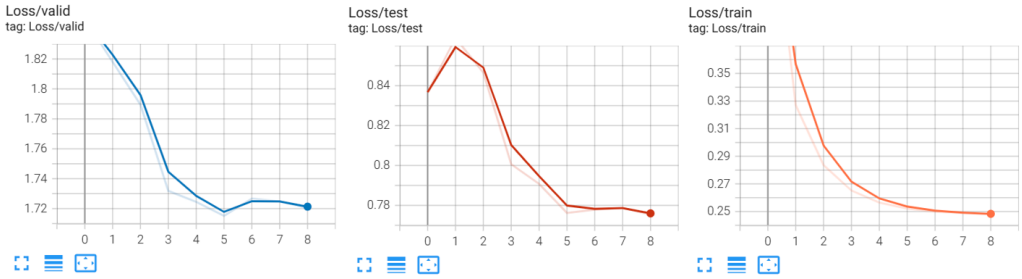
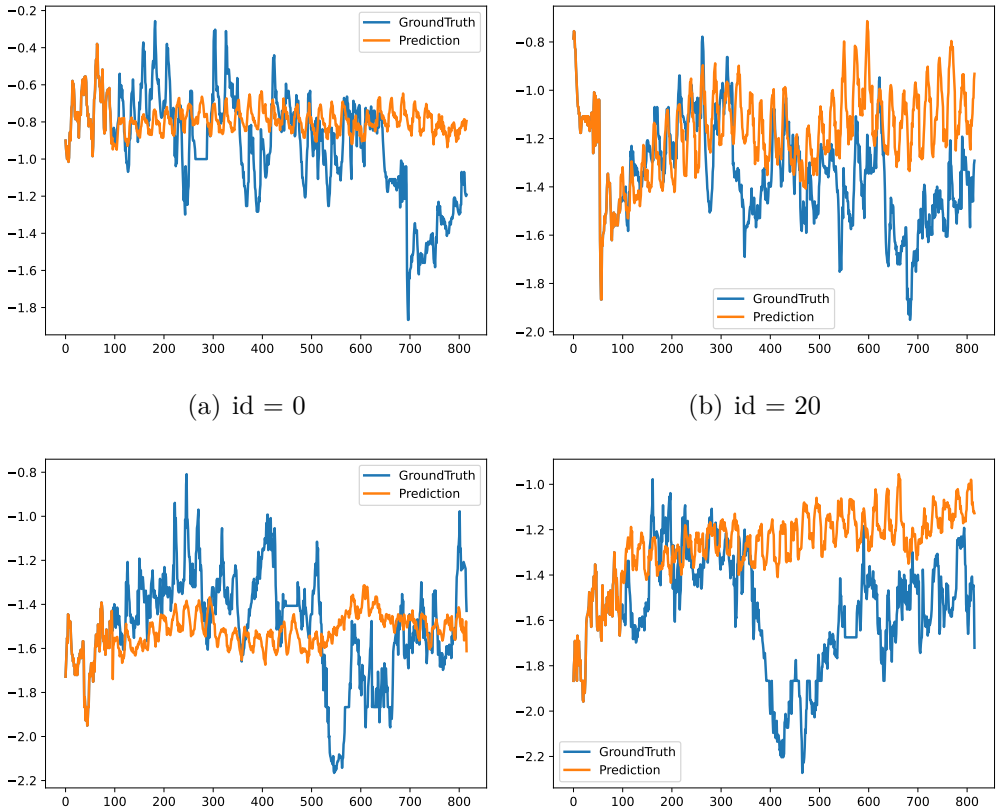


图 1: 模型在验证集、测试集和训练集上的 loss 变化曲线



(a) id = 0

(b) id = 20

(c) id = 40

(d) id = 60

图 2: 标准模型的部分预测结果

合能力有限。

取在验证集上表现最好的模型，在测试集上计算误差，结果为：

- MSE: 0.775672197341919
- MAE: 0.648556411266327

1.2 Task2: Technical Details

在回归预测任务中，为保证模型是自回归的，我们在训练过程中会在预测值及其之后的内容上添加 mask，以此来强制要求模型在学习时只能看到预测值之前的内容。否则，模型在训练时可能会“作弊”，即错误地使用了在推理过程本不可见的未来信息，最终导致模型无法学习到某些可泛化的数据规律。

实现 mask 的方法很简单，只需要在计算注意力时，在 softmax 算子前将 attention 张量中预测值及其之后内容的相应位置上的值设置为-inf 即可(在本例中即为矩阵的上三角部分)。代码实现如下：

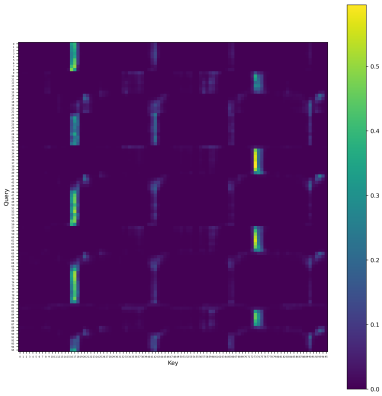
```
if attn_mask is not None:
    attn = attn.masked_fill(th.logical_not(attn_mask), -th.inf)
attn = nn.functional.softmax(attn, dim=3)
```

1.3 Task3: Attention Visualization

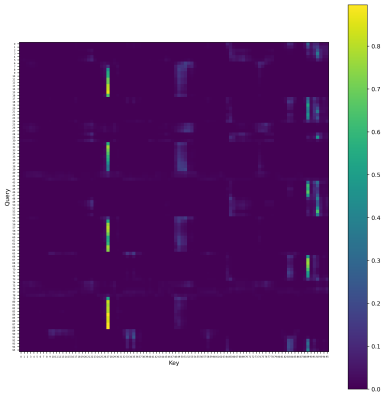
在 Transformer 模型中，我们截取出最后一层 Encoder 的 attention 值，将其可视化为热力图的形式，结果如图 3 所示。

图 3 的热力图和图 2 的预测结果是相对应的。我们发现，Transformer 模型确实学习到了一些相关性特征。具体而言，在热力图中，每间隔一定距离，key 和 query 就具有较大的 attention 值，说明它们具有较强的相关性。

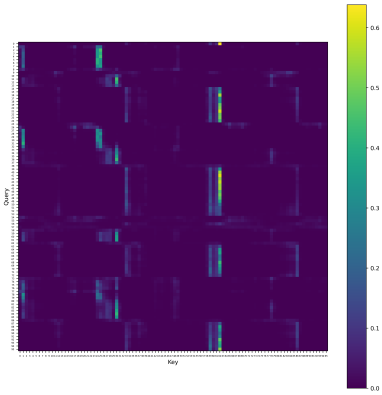
这意味着我们的预测对象——electricity，体现出了明显的周期性：每隔一定时长（约为 24h）的历史数据对于当前值的影响较为显著，而其他时



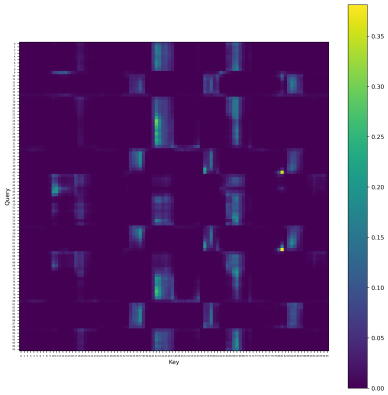
(a) id = 0



(b) id = 20



(c) id = 40



(d) id = 60

图 3: 注意力的热力图可视化结果

间的历史数据与当前值的相关性则相对较小。

1.4 Task4: Extra Techniques

在使用标准 Transformer 模型时，我们发现训练时间较长。这是因为标准 Transformer 模型的复杂度正比于与序列长度的平方，在输入序列的规模增大时，效率会迅速降低。

参考相关论文，我采用了基于 Local Attention 的改进方式。[2] Local Attention 是 Sparse Attention 的一种实现，在训练和推理时只对临近的 48 个元素（即本例中最大序列长度的一半）计算 attention，可以有效地提高 Transformer 模型的训练与推理效率。

相关代码实现参见./Transformer/layers/attention.py 的 106-109 行。

训练过程中，模型在训练集、测试集和验证集上的 loss 变化曲线如图 2 所示。

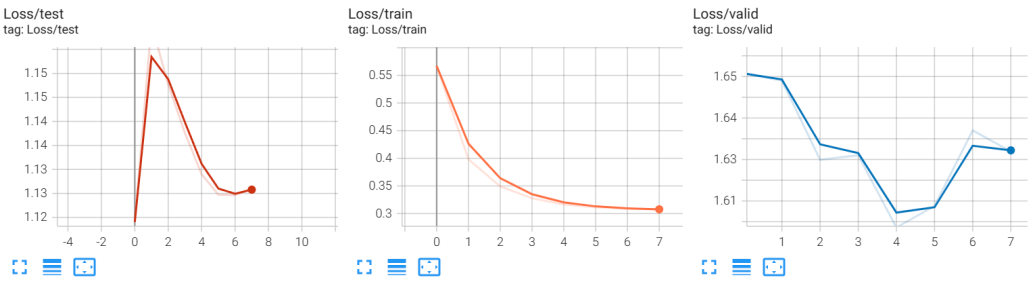


图 4: 改进模型在验证集、测试集和训练集上的 loss 变化曲线

如表 1 所示，我们尝试对比改进前后模型的训练用时和测试误差。

表 1: 标准 Transformer 模型和改进 Transformer 模型的对比

model	Time Cost	MSE	MAE
Standard Transformer	20 min	0.776	0.648
Sparse Transformer	12 min	0.928	0.713

结果显示，在引入了 Local Attention 后，虽然模型的测试误差有微幅的提高，但训练的时间消耗显著降低，降低幅度将近一倍。综上，通过改进 attention 的计算方式，我们成功地提高了 Transformer 模型的效率，同时控制了精度损失在可接受的范围之内。

2 Generative Adversarial Networks (GAN)

3 Diffusion Models

运行环境为：Ubuntu 22.04.3 LTS，Python 3.9.18，Tensorflow 2.15.0，PyTorch 2.1.2。

3.1 原理推导

1. 证明：在 $t \to \infty$ 时，x_t 收敛至服从标准正态分布。

   我们引入服从正态分布的噪声参数 $\epsilon \sim \mathcal{N}(0, I)$，则前向扩散过程可以重参数为如下形式：

$$x_t = \sqrt{1-\beta_t} x_{t-1} + \sqrt{\beta_t}\epsilon \tag{1}$$

   记 $\alpha_t = 1 - \beta_t$，有：

$$\begin{aligned} x_t =& \sqrt{1-\beta_t} x_{t-1} + \beta_t \epsilon \\ =& \sqrt{\alpha_t} x_{t-1} + \sqrt{1-\alpha_t}\epsilon \\ =& \sqrt{\alpha_t}(\sqrt{\alpha_{t-1}} x_{t-2} + \sqrt{1-\alpha_{t-1}}\epsilon') + \sqrt{1-\alpha_t}\epsilon \\ =& \sqrt{\alpha_t\alpha_{t-1}} x_{t-2} + (\sqrt{\alpha_t - \alpha_t\alpha_{t-1}}\epsilon' + \sqrt{1-\alpha_t}\epsilon) \end{aligned}$$

   其中 ϵ 和 ϵ' 是互相独立的标准正态分布随机变量，有：

$$(\sqrt{\alpha_t - \alpha_t\alpha_{t-1}}\epsilon' + \sqrt{1-\alpha_t}\epsilon) \sim \mathcal{N}(0, (1-\alpha_t\alpha_{t-1})I)$$

   于是：

$$x_t = \sqrt{\alpha_t\alpha_{t-1}} x_{t-2} + (\sqrt{\alpha_t - \alpha_t\alpha_{t-1}}\epsilon' + \sqrt{1-\alpha_t}\epsilon)$$

$$
\begin{aligned}
&=\sqrt{\alpha_t\alpha_{t-1}}x_{t-2}+\sqrt{1-\alpha_t\alpha_{t-1}}\epsilon \\
&=\ldots\ldots \\
&=\sqrt{\prod_{i=1}^{t}\alpha_i}\cdot x_0+\sqrt{1-\prod_{i=1}^{t}\alpha_i}\cdot\epsilon
\end{aligned}
$$

在扩散模型中，我们选取了 $\beta_t = c \in (0,1]$，于是：$\alpha_t = 1-\beta_t$ 的上界应严格小于 1。因此，在 $t\to\infty$ 时，$\prod_{i=1}^{t}\alpha_i \to 0$，即有：

$$
x_t \to \epsilon \sim \mathcal{N}(0, I) \tag{2}
$$

2. 证明：后验分布 $q(x_{t-1}|x_t, x_0)$ 服从正态分布。

使用 Bayes 公式：

$$
\begin{aligned}
q(x_{t-1}|x_t, x_0) =& \frac{q(x_{t-1}, x_t|x_0)}{q(x_t|x_0)} \\
=& \frac{q(x_t|x_{t-1}, x_0)q(x_{t-1}|x_0)}{q(x_t|x_0)} \\
=& \frac{q(x_t|x_{t-1})q(x_{t-1}|x_0)}{q(x_t|x_0)}
\end{aligned}
$$

记 $\overline{\alpha}_t = \prod_{i=1}^{t}\alpha_i$，我们已经证明：$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\overline{\alpha}_t}, \sqrt{1-\overline{\alpha}_t}I)$，故：

$$
\begin{aligned}
&q(x_{t-1}|x_t, x_0) \\
=&\frac{q(x_t|x_{t-1})q(x_{t-1}|x_0)}{q(x_t|x_0)} \\
\propto&\exp\{\frac{(x_t-\sqrt{\alpha_t}x_{t-1})^2}{2(1-\alpha_t)}+\frac{(x_{t-1}+\sqrt{\overline{\alpha}_{t-1}})^2}{2(1-\overline{\alpha}_{t-1})}-\frac{(x_t-\sqrt{\overline{\alpha}_t})^2}{2(1-\overline{\alpha}_t)}\} \\
=&\exp\{-\frac{1}{2}[\frac{1-\overline{\alpha}_t}{(1-\alpha_t)(1-\overline{\alpha}_{t-1})}x_{t-1}^2-2(\frac{\sqrt{\alpha_t}x_t}{1-\alpha_t}+\frac{\sqrt{\overline{\alpha}_{t-1}}x_0}{1-\overline{\alpha}_{t-1}})x_{t-1}+C]\}
\end{aligned}
$$

其中 $C = C(x_t, x_0)$，是与 x_{t-1} 无关的常量。于是：

$$
q(x_{t-1}|x_t, x_0)
$$

$$\sim \exp\{-\frac{1}{2}[\frac{(x_{t-1}-\frac{\sqrt{\alpha_t}(1-\overline{\alpha}_{t-1})x_t+\sqrt{\overline{\alpha}_{t-1}}(1-\alpha_t)x_0}{1-\overline{\alpha}_t})^2}{(\sqrt{\frac{(1-\alpha_t)(1-\overline{\alpha}_{t-1})}{1-\overline{\alpha}_t}})^2}]\}$$

$$\sim \mathcal{N}(x_{t-1};\frac{\sqrt{\alpha_t}(1-\overline{\alpha}_{t-1})x_t+\sqrt{\overline{\alpha}_{t-1}}(1-\alpha_t)x_0}{1-\overline{\alpha}_t},\frac{(1-\alpha_t)(1-\overline{\alpha}_{t-1})}{1-\overline{\alpha}_t}I)$$

对 $q(x_{t-1}|x_t,x_0)$ 归一化，:

$$q(x_{t-1}|x_t,x_0)$$
$$=\mathcal{N}(x_{t-1};\frac{\sqrt{\alpha_t}(1-\overline{\alpha}_{t-1})x_t+\sqrt{\overline{\alpha}_{t-1}}(1-\alpha_t)x_0}{1-\overline{\alpha}_t},\frac{(1-\alpha_t)(1-\overline{\alpha}_{t-1})}{1-\overline{\alpha}_t}I)$$

综上，$q(x_{t-1}|x_t,x_0)$ 服从正态分布 $\mathcal{N}(\tilde{u}_t,\tilde{\beta}_t I)$，其中均值 $\tilde{u}_t$、方差 $\tilde{\beta}_t$ 为:

$$\tilde{u}_t=\frac{\sqrt{\alpha_t}(1-\overline{\alpha}_{t-1})x_t+\sqrt{\overline{\alpha}_{t-1}}(1-\alpha_t)x_0}{1-\overline{\alpha}_t} \tag{3}$$

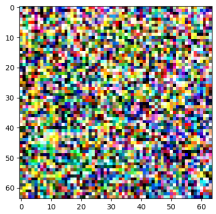
$$\tilde{\beta}_t=\frac{(1-\alpha_t)(1-\overline{\alpha}_{t-1})}{1-\overline{\alpha}_t} \tag{4}$$

3.2 DDPM

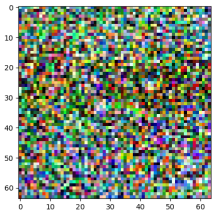
调整 classifier_sample.sh 中的 diffusion_steps 参数，基于 DDPM 生成的图像如图 5 所示。同时，我们还记录了生成图片在 ImageNet 中的类别名称，以供参考。

可以看出，DDPM 能够成功生成符合一般认知逻辑的图片，其内容也与 ImageNet 中的类别名称相符合。

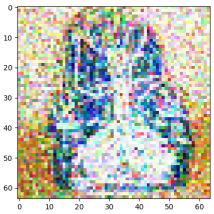
通过调整 diffusion_steps 参数，我们发现：在采样步数小于 64 步时，DDPM 生成的图片开始变得模糊；在采样步数小于 8 步时，DDPM 生成的图片已无法辨识。而当采样步数过大时（大于 256 步），图片的清晰度并没有较为明显的提升。因此，考虑到生成效率的问题，我们认为取 diffusion_steps 为 256-512 较为合适。



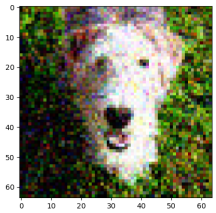
(a) steps=2, Persian Cat



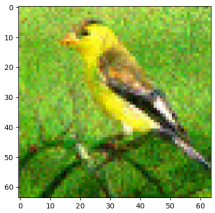
(b) steps=4, Nile Crocodile



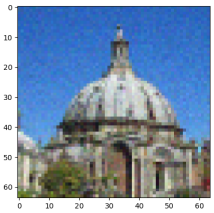
(c) steps=8, Backpack



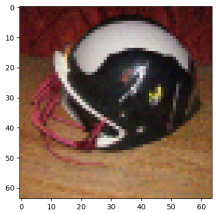
(d) steps=16, Fox Terrier



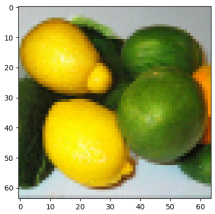
(e) steps=32, Goldfinch



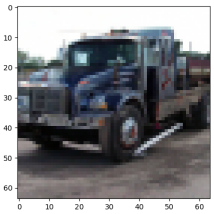
(f) steps=64, Dome



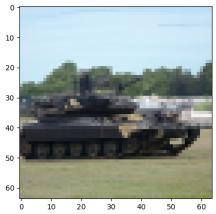
(g) steps=128, Helmet



(h) steps=256, Lemon

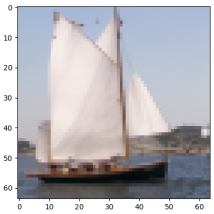


(i) steps=512, Tow Truck



(j) steps=1000, Tank

(k) steps=1024, Hen



(l) steps=2048, Yawl

图 5: DDPM 在不同 diffusion_steps 参数下生成的图片示例

3.3 DDIM

仿造 GaussianDiffusion 类中 p_sample 方法和 p_mean_variance 方法，并参考课件 86 页给出的公式，我们补全了 DDIM 的 sampler。[3] 源代码参见./Diffusion/guided_diffusion/gaussian_diffusion.py 中的 ddim_sample 方法和 p_mean_variance_ddim 方法。另外，注意到在实际调用模型时，p_mean_variance_ddim 方法还需要在子类 SpacedDiffusion 重写，此部分参见./Diffusion/guided_diffusion/respace.py 中 SpacedDiffusion 类的实现。

调整 diffusion_steps 参数，基于 DDIM 生成的图像及其类别名称如图 6 所示。

可以看出，DDIM 的生成效果同样较好。在采样步数大于 128 步时，DDIM 和 DDPM 的生成效果几乎没有区别，都可以生成与类别名称相符合的低噪图像。

注意到，在采样步数小于 128 步时（16 步、32 步和 64 步），在 DDIM 的生成图像中开始出现明显的噪点。对比而言，在同样的采样步数下，DDPM 的生成图像的质量同样有所下降。二者在直观表现上存在一定的区别。DDIM 的生成图像整体上仍然较为清晰，但噪点更多、更密集；而在 DDPM 的生成图像中并没有观察到明显的噪点，但整体的清晰度更低。

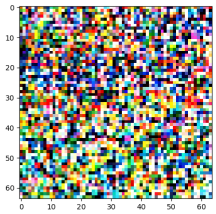
进一步降低采样步数，在采样步数小于 16 时，DDPM 的生成图像已经几乎无法辨识。而在 DDIM 的生成图像中，虽然存在大量噪点，但仍然能够大致辨认出图像的类别。

3.4 CGDM

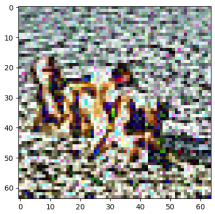
设置 class_index=847，我们希望模型生成类别为“坦克”的图片。

固定 diffusion_steps=512，使用 DDPM，我们调整 classifier_scale 参数的值，观察模型生成图片的效果，结果如图 7-图 11 所示。

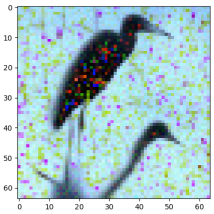
在 classifier_scale 较大时（为 100 和 10 时），模型对类别的要求较高，模型生成的图像几乎都是晴朗天气下在开阔地带的带有火炮的履带式坦克，



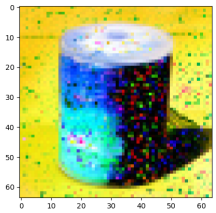
(a) steps=2, Butcher Shop



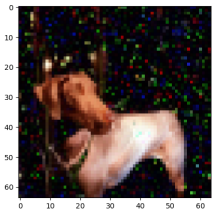
(b) steps=4, Welsh Spaniel



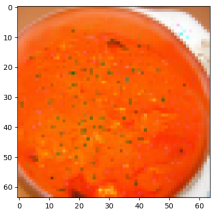
(c) steps=8, Heron



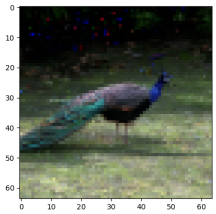
(d) steps=16, Oil Filter



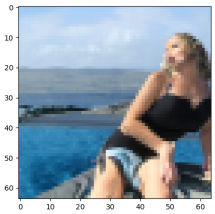
(e) steps=32, Weimaraner



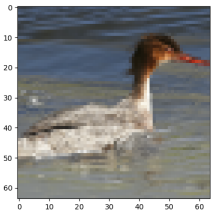
(f) steps=64, Frypan



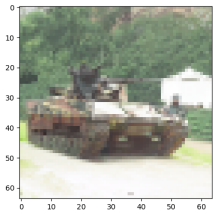
(g) steps=128, Peacock



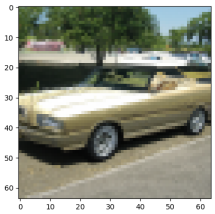
(h) steps=256, Maillot



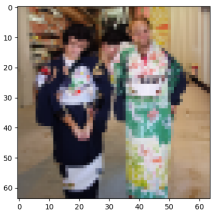
(i) steps=512, Merganser



(j) steps=1000, Tank



(k) steps=1024, Convertible



(l) steps=2048, Kimono

图 6: DDIM 在不同 diffusion_steps 参数下生成的图片示例

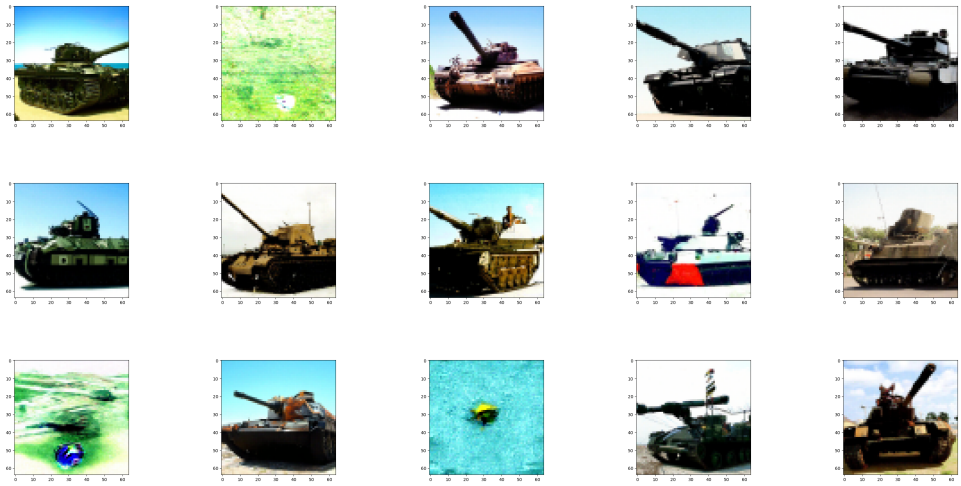


图 7: CGDM 的生成图像, 指定生成类别为 847(Tank), classifier_scale=100

图 8: CGDM 的生成图像：指定生成类别为 847(Tank), classifier_scale=10

图 9: CGDM 的生成图像：指定生成类别为 847(Tank), classifier_scale=1

图 10: CGDM 的生成图像：指定生成类别为 847(Tank), classifier_scale=1e-2

图 11: CGDM 的生成图像：指定生成类别为 847(Tank), classifier_scale=1e-4

且朝向均为向左或向右。此时，模型的类别相关性较高，但多样性不足。另外，我们注意到在 classifier_scale=100 时，模型的生成质量降低，生成的部分图像完全崩坏，不能识别。

在 classifier_scale 较小时（为 1e-4 和 1e-2 时），模型对类别的要求较低，模型生成的图像中，既有带有火炮的履带式坦克，也有自行火炮、装甲车、坦克底盘、吉普车、拖拉机等载具，在朝向上也不仅仅局限于左、右两个方向，且图像的背景也更为丰富，除晴朗开阔地带环境外，还有黑夜环境、沙尘环境等。此时，模型的多样性较高，但生成的图片并不严格地属于坦克类别。

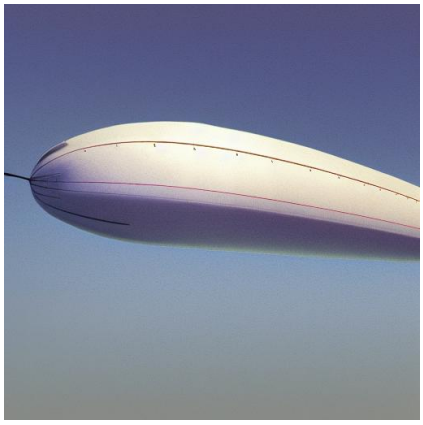
在 classifier_scale 较适中时（为 1 时），模型能够在多样性和类别相关性间取得平衡，生成的图像具有一定的多样性，同时也保证了图像内容基本为带有火炮的履带式坦克。

3.5 Stable Diffusion

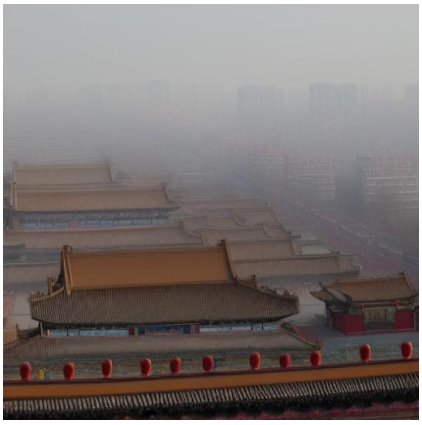
如图 12 所示，我找到了 6 个能使 Stable Diffusion 生成不合理图片的文本提示。

(a) A little kid with a pair of scissors

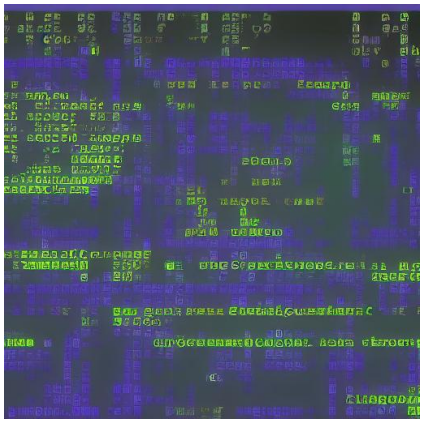
(b) Four round swimming pools



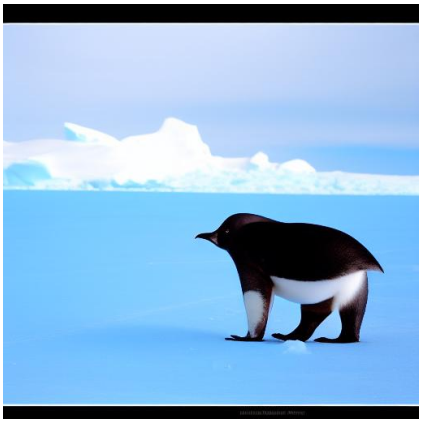
(c) Zeppelin Airship



(d) Beijing in 30,000 B.C.



(e) A piece of Python code



(f) Animals of Antarctica

图 12: 失败的 Stable Diffusion 提示示例

(a) 在生成图像中，剪刀只有一半，且小孩的另一只手浮空，非常诡异。

(b) 要求生成四个游泳池，但图片中只有两个。

(c) 生成的齐柏林飞艇没有客舱，其形状也并非正常的椭球形。

(d) 公元前 30000 年，北京不可能有宫殿式的建筑。

(e) 要求生成一段 Python 代码，但完全无法辨识模型的生成内容。

(f) 生成了一只混合了北极熊和企鹅特征的奇怪生物。

Notes

[1]Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, & R. Garnett (Eds.), Advances in Neural Information Processing Systems 30 (pp. 5998–6008).

[2]Child, R., Gray, S., Radford, A., & Sutskever, I. (2019). Generating long sequences with sparse transformers. arXiv preprint arXiv:1904.10509

[3]Song, J., Meng, C., & Ermon, S. (2021). Denoising diffusion implicit models. In Proceedings of the 9th International Conference on Learning Representations (pp. 1-18)